

SZAKDOLGOZAT



MISKOLCI EGYETEM

SPAM/HAM üzenetek osztályozása neurális hálók és nagy nyelvi modellek (LLM) segítségével

Készítette:

Ferencsik Róbert

Programtervező informatikus

Témavezető:

Dr. Baksáné Dr. Varga Erika

MISKOLC, 2026

MISKOLCI EGYETEM

Gépészmérnöki és Informatikai Kar

Alkalmazott Matematikai Tanszék

Szám:

SZAKDOLGOZAT FELADAT

Ferencsik Róbert (BQLOTW) BSc programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: gépi tanulás, klasszifikáció

A szakdolgozat címe: SPAM/HAM üzenetek és forráskódok osztályozása neurális hálók és nagy nyelvi modellek (LLM) segítségével

A feladat részletezése:

A SPAM/HAM klasszifikációs feladat lényege annak eldöntése, hogy egy adott üzenet *spam* (kéretlen, nem kívánt tartalom), vagy *ham* (nem spam, hasznos tartalom). A probléma jól vizsgálható gépi tanulási és mélytanulási módszerekkel; különösen az LSTM (Long Short-Term Memory) architektúrák alkalmasak szekvenciális adatok—így szövegek—feldolgozására.

A szakdolgozatban a hallgató először a SPAM/HAM osztályozást valósítja meg, majd a megközelítést kiterjeszti forráskódokra: a programkódokat *malware* vagy *benign* kategóriába kell sorolni.

A modern nagy nyelvi modellek (LLM) lehetőséget adnak mind a természetes nyelvi szövegek, mind a programkódok szemantikai feldolgozására. A dolgozat célja az LSTM- és LLM-alapú megközelítések *összehasonlítása* és *teljesítményük értékelése* (megfelelő validációs és tesztelési protokoll mellett).

Témavezető(k): Dr. Baksáné Dr. Varga Erika, egyetemi docens

Konzulens(ek):

A feladat kiadásának ideje: 2025. szeptember 9.

.....
szakfelelős

EREDETISÉGI NYILATKOZAT

Alulírott **Ferencsik Róbert**; Neptun-kód: BQL0TW a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírással igazolom, hogy *SPAM/HAM üzenetek és forráskódok osztályozása neurális hálók és nagy nyelvi modellek (LLM) segítségével* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szó szerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, év hó nap

.....
Hallgató

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- | | |
|------------------------------|-----------------------------|
| témavezető (dátum, aláírás): | konzulens (dátum, aláírás): |
| | |
| | |
| | |

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- dátum szakfelelős

- Miskolc,
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés	1
2. Konceptió	3
2.1. A fejezet célja	3
2.2. Tartalom és felépítés	3
2.3. Amit csak említés szintjén érdemes szerepeltetni	3
3. Tervezés	5
3.1. Táblázatok	5
3.2. Ábrák	5
3.3. További környezetek	5
4. Megvalósítás	7
5. Tesztelés	9
6. Összefoglalás	10
Források	11

1. fejezet

Bevezetés

Az információs technológiák rohamos fejlődése alapjaiban alakította át a kiberbiztonság területét. Az új technológiák megjelenésével ugyan jelentősen bővült a védekezés eszköztára, ez azonban korántsem jelenti a fenyegetések megszűnését vagy csökkenését. A digitális térben zajló „küzdelem” inkább egy folyamatosan változó, dinamikus folyamatként írható le, ahol a támadási és védekezési módszerek egymással párhuzamosan fejlődnek. Az internet széles körű elterjedése egy korábbi jelentős fordulópontot jelentett a kiberbiztonság történetében, míg napjainkban a mesterséges intelligencia új eszközöket biztosít mind a támadók, mind a védekezők számára.

A köztudatban élő kép, miszerint a kiberbiztonság valós idejű „programozói párbajként” értelmezhető, nem tükrözi a valóság összetettségét. A gyakorlatban a hangsúly sokkal inkább a már működő rendszerekben található sérülékenységek feltárásán és kezelésén van. Egy informatikai rendszer biztonsága nem csupán a forráskód minőségétől függ, hanem számos egyéb tényező együttes hatásának eredménye. Ilyen tényezők például a titkosítás hiánya, a nem megfelelő jogosultságkezelés, a hibás konfigurációk, és a legfontosabb az emberi tényező.

Az egyik leggyakoribb támadási forma az **adathalászat** (phishing). Az ilyen típusú támadások során a támadók megtévesztő, gyakran hitelesnek tűnő üzenetek segítségével próbálnak érzékeny adatokat megszerezni a felhasználóktól. Az adathalász e-mailek különösen veszélyesek, mivel kihasználják a felhasználók figyelmetlenségét, illetve döntéshozatali korlátait. Mindez rámutat arra, hogy a kizárólag felhasználói döntésekre építő védekezés nem tekinthető elegendőnek. Illetve a nem kívánt és egyéb e-mailek osztályozása a biztonság mellett a levelezési fiók rendszerezettségét és a hatékonyságot is növeli a mindennapi és a céges környezetekben.

Ebből következően a modern kiberbiztonsági megoldások egyik kulcseleme az automatizált védelem alkalmazása. Az e-mail szolgáltatók fejlett szűrőrendszereket használnak annak érdekében, hogy a nem kívánt vagy kártékony üzeneteket még a felhasználóhoz való eljutás előtt azonosítsák és elkülönítsék. A beérkező üzenetek nagy mennyisége és változatossága miatt ezek a rendszerek jellemzően gépi tanulási módszereken alapulnak, amelyek képesek alkalmazkodni az új mintázatokhoz és folyamatosan fejlődő támadási technikákhoz.

Ezt a folyamatot klasszifikációnak, osztályba sorolásnak tekintjük. Mely lehet egy, kettő vagy akár több osztályú probléma is. Gépi módszerek számos fajtája alkalmazható erre a feladatra, és ugyancsak sokféle adatból lehet döntést hozni egy e-mail kéretlen voltáról. A szöveg alapú megközelítések mellett a metaadatokból, hálózati jellemzőkből és viselkedési mintákból automata vagy fél automata módon információ kinyerő

modellek és ezek konkrét megvalósítása végtelen módot nyújt a feladat megoldására.

Az egyik megközelítés a **kéretlen levél** (spam) és **valódi levél** (ham) kategóriákat választja két csoportra természetes nyelvű szekvenciák alapján. Ezen dolgozat ezt veszi alapul, és vizsgálja **mélytanulási** (Deep Learning) módszerekkel. Időben vagy térben folytonos sorrendű adatok feldolgozására már limitáltabb az eszközeink tárháza. Természetes nyelv szekvencia feldolgozására különösen alkalmasak a **hosszú-rövid távú memória** (Long Short-Term Memory, LSTM) architektúrán alapuló visszacsatolt neurális hálózatok, melyek a nyelvben előforduló statisztikai mintázatokat tanulják meg. Így képességet szerezve a következő betű, szórészlet vagy szó valószínűségi eloszlásának megbecsülésére a megelőző kontextus alapján. Ebben az értelemben a nyelvet egy sorozat becslésként modellezi, ahol minden új nyelvtani egység az előzőek alapján kerül meghatározásra.

A legmodernebb módszer elméleti szinten hasonlóan működik, de a modell szerkezete nagyban eltérő. Az új **nagy nyelvi modellek** (Large Language Model, LLM) melyek a transformer architektúrán alapulnak, előre tanítottak hatalmas mennyiségű szöveges adaton, így előzetes tudást nyerve az adott nyelv(ek)ről. Szintén adott sorozat alapján végeznek predikciót a következő egységre, azonban sokkal erőteljesebbek, kevésbé érzékenyek a zajos szövegekre (nem várt karakterek, helytelen szavak, számok stb.) így kevesebb, illetve közel nulla előfeldolgozást igényelnek tanítás és predikció közben.

A dolgozat során egy hosszú-rövid távú memória felépítése és betanítása kerül bemutatásra, az adatok előfeldolgozásával együtt. Ez a technika még érzékeny a zajos adatokra, így a tisztítás és helyes előfeldolgozás nagy hangsúlyt kapott, habár ennek eredménye számszerű mutatókkal kerül demonstrálásra, nem pedig a látványosabb működésközbeni levélszemét elválasztásával. A módszerek és mérőszámok kifejtésre kerülnek a későbbiekben. Az alapvető felvetés azt vizsgálja, hogy mennyi adatot érdemes felhasználni egy ilyen architektúra tanítására, a sikeres adat tisztítás és modell paraméterek megfelelő beállítása mellett. Ezt egy fokozódó tanító adaton de azonos architektúrán való modellek tanításával kerül demonstrálásra. Az így keletkezett modellek kiértékeléséből egy **tanító görbe** (Learning Curve) kerül kirajzolásra.

A bevezetést követően a következő fejezetek ismertetik a természetes nyelvfeldolgozás és a szöveglaszifikáció elméleti alapjait. Ezt követi az adatkészlet és az előfeldolgozási lépések részletes bemutatása, majd a modell architektúrájának és tanítási folyamatának ismertetése. A kísérleti eredmények bemutatása után a dolgozat záró fejezete összegzi a legfontosabb megállapításokat, kitér a korlátokra, valamint javaslatokat fogalmaz meg a további kutatási irányokra.

2. fejezet

Koncepció

2.1. A fejezet célja

Ez a fejezet még nem a saját eredményekkel foglalkozik, hanem bemutatja, mi a problémakör, milyen módszerekkel, milyen eredményeket sikerült elérni eddig másoknak.

A hivatkozások jelentős része ehhez a fejezethez szokott kötődni. (Egy hivatkozás például így néz ki [1].) Itt lehet bemutatni a hasonló alkalmazásokat.

2.2. Tartalom és felépítés

A fejezet tartalma témától függően változhat. Az alábbiakat attól függően különböző arányban tartalmazhatják.

- Irodalomkutatás. Amennyiben a dolgozat egy módszer kidolgozására, kifejlesztésére irányul, akkor itt lehet részletesen végignézni (módszertani vagy időrendi bontásban), hogy az eddigiekben milyen eredmények születtek a témakörben.
- Technológia. Mivel jellemzően kutatásról vagy szoftverfejlesztésről van szó, ezért annak a jellemző elemeit, technikai részleteit itt kell bemutatni. Ez tehát egy módszeres bevezetés ahhoz, hogy ha valaki nem jártas a témakörben, akkor tudja, hogy a dolgozat milyen aktuálisan elérhető eredményeket, eszközöket használt fel.
- Piackutatás. Bizonyos témáknál új termék vagy szolgáltatás kifejlesztése a cél. Ekkor érdemes annak alaposan utánanézni, hogy aktuálisan milyen eszközök érhetők el a piacon. Ez szoftverek esetében a hasonló alkalmazások bemutatását, táblázatos formában történő összehasonlítását jelentheti. Szerepelhetnek képek és észrevételek a viszonyításként bemutatott alkalmazásokhoz.
- Követelmény specifikáció. Külön szakaszban érdemes részletesen kitérni az elkészítendő alkalmazással kapcsolatos követelményekre. Ehhez tartozhatnak forgatókönyvek (*scenario*-k). A szemléletesség kedvéért lehet hozzájuk képernyőkép vázlatokat is készíteni, vagy a használati eseteket más módon szemléltetni.

2.3. Amit csak említés szintjén érdemes szerepeltetni

Az olvasóról annyit feltételezhetünk, hogy programozásban valamilyen szinten járatos, és a matematikai alapfogalmakkal sem ebben a dolgozatban kell megismertetni. A spe-

ciális eszközök, programozási nyelvek, matematikai módszerek és jelölések persze jó, hogy ha említésre kerülnek, de nem kell nagyon belemenni a közismertnek tekinthető dolgokba.

3. fejezet

Tervezés

Itt kezdődik a dolgozat lényegi része, úgy érte, hogy a saját munka bemutatása. Jellemzően ebben szerepelni szoktak blokkdiagramok, a program struktúrájával foglalkozó leírások. Ehhez célszerű UML ábrákat (például osztály- és szekvenciadiagramokat) használni.

Amennyiben a dolgozat inkább kutatás jellegű, úgy itt lehet konkretizálni a kutatási módszertant, a kutatás tervezett lépéseit, az indoklást, hogy mit, miért és miért pont úgy érdemes csinálni, ahogyan az a későbbiekben majd részletezésre kerül.

Ebben a fejezetben az implementáció nem kell, hogy túl nagy szerepet kapjon. Ez még csak a tervezési fázis. (Nyilván ha olyan a téma, hogy magának az implementációnak a módjával foglalkozik, adott formális nyelvet mutat be, úgy a kódpéldákat már innen sem lehet kihagyni.)

3.1. Táblázatok

Táblázatokhoz a `table` környezetet ajánlott használni. Erre egy minta a 3.1. táblázat. A hivatkozáshoz az egyedi `label` értéke konvenció szerint `tab:` prefixszel kezdődik.

3.1. táblázat: Minta táblázat. A táblázat felirata a táblázat felett kell legyen!

a	b	c
1	2	3
4	5	6

3.2. Ábrák

Ábrákat a `figure` környezettel lehet használni. A használatára egy példa a 3.1. ábrában látható. Az `includegraphics` parancsba Az ábrák felirata az ábra alatt kell legyen. Az ábrák hivatkozásához használt nevet konvenció szerint `fig:-`el célszerű kezdeni.

3.3. További környezetek

A matematikai témájú dolgozatokban szükség lehet tételek és bizonyításaik megadására. Ehhez szintén vannak készen elérhető környezetek.



3.1. ábra: A Miskolci Egyetem címere.

3.1. definíció. Ez egy definíció

3.2. lemma. *Ez egy lemma*

3.3. tétel. *Ez egy tétel*

Bizonyítás. Ez egy bizonyítás

□

3.4. következmény. *Ez egy tétel*

3.5. megjegyzés. Ez egy megjegyzés

3.6. példa. Ez egy példa

4. fejezet

Megvalósítás

Ez a fejezet mutatja be a megvalósítás lépéseit. Itt lehet az esetlegesen előforduló technikai nehézségeket említeni. Be lehet már mutatni a program elkészült részeit.

Meg lehet mutatni az elkészített programkód érdekesebb részeit. (Az érdekesebb részek bemutatására kellene szorítkozni. Többségében a szöveges leírásnak kellene benne lennie. Abból lehet kiindulni, hogy a forráskód a dolgozathoz elérhető, azt nem kell magába a dolgozatba bemásolni, elegendő csak behivatkozni.)

A dolgozatban szereplő forráskódrészletekhez külön vannak programnyelvenként stílusok. Python esetében például a 4.1. programkódban látható egy formázott kódrészlet.

Programkód 4.1. Python példa

```
import sys

if __name__ == '__main__':
    pass
```

A stílusfájlok a **styles** jegyzékben találhatók. A stílusok között szerepel még C++, Java és Rust stílusfájl. Ezek használatához a **dolgozat.tex** fájl elején **usepackage** paranccsal hozzá kell adni a stílust, majd a stílusfájl nevével megegyező környezetet lehet használni. További példaként C++ forráskód esetében ez így szerepel.

```
#include <iostream>

class Sample : public Object
{
    // An empty class definition
}
```

Stílusfájlokból elegendő csak annyit meghagyni, amennyire a dolgozatban szükség van. Más, C szintaktikájú nyelvekhez (mint például a JavaScript és C#) a Java vagy C++ stílusfájlok átszerkesztésére van szükség. (Elegendő lehet csak a fájlnevet átírni, és a fájlban a környezet nevét.)

Nyers adatok, parancssori kimenetek megjelenítéséhez a **verbatim** környezetet lehet használni.

```
$ some commands with arguments
1 2 3 4 5
$ _
```

A kutatás jellegű témáknál ez a fejezet gyakorlatilag kimaradhat. Helyette inkább a fő vizsgálati módszerek, kutatási irányok kaphatnak külön-külön fejezeteket.

5. fejezet

Tesztelés

A fejezetben be kell mutatni, hogy az elkészült alkalmazás hogyan használható. (Az, hogy hogyan kell, hogy működjön, és hogy hogy lett elkészítve, az előző fejezetekben már megtörtént.)

Jellemzően az alábbi dolgok kerülhetnek ide.

- Tesztfuttatások. Le lehet írni a futási időket, memória és tárigényt.
- Felhasználói kézikönyv jellegű leírás. Kifejezetten a végfelhasználó szempontjából lehet azt bemutatni, hogy mit hogy lehet majd használni.
- Kutatás kapcsán ide főként táblázatok, görbék és egyéb részletes összesítések kerülhetnek.

6. fejezet

Összefoglalás

Hasonló szerepe van, mint a bevezetésnek. Itt már múltidőben lehet beszélni. A szerző saját meglátása szerint kell összegezni és értékelni a dolgozat fontosabb eredményeit. Meg lehet benne említeni, hogy mi az ami jobban, mi az ami kevésbé jobban sikerült a tervezettnél. El lehet benne mondani, hogy milyen további tervek, fejlesztési lehetőségek vannak még a témával kapcsolatban.

Források

- [1] James H Coombs, Allen H Renear és Steven J DeRose. „Markup systems and the future of scholarly text processing”. *Communications of the ACM* 30.11 (1987), 933–947. old.

CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy Markdown fájlként kimentve).